1. Create a class called Song in your preferred OOP language with private properties title and artist. Add public getter and setter methods to access and modify these properties, then create an object and update its title.

```
class Song {
   private String title;
   private String artist;

   public void setTitle(String title) {
      this.title = title;
   }

   public String getTitle() {
      return title;
   }

   public void setArtist(String artist) {
      this.artist = artist;
   }

   public String getArtist() {
      return artist;
   }
}

public class Main {
   public static void main(String[] args) {
      Song s = new Song();
```

```
        s.setTitle("Perfect");
        s.setArtist("Ed Sheeran");

        System.out.println("Old Title: " + s.getTitle());

        s.setTitle("Shape of You");

        System.out.println("Updated Title: " + s.getTitle());
        System.out.println("Artist: " + s.getArtist());
    }
}
```

Output:

```
Old Title: Perfect
Updated Title: Shape of You
Artist: Ed Sheeran
```

2. Build a class called InstaStory with a protected property storyViews. Create a subclass called SponsoredStory that can access and display the storyViews value.

```
class InstaStory {
    protected int storyViews = 500;
}

class SponsoredStory extends InstaStory {
    public void displayViews() {
        System.out.println("Story Views: " + storyViews);
    }
}
```

```
public class Main {
  public static void main(String[] args) {
    SponsoredStory s = new SponsoredStory();
    s.displayViews();
  }
}
```

Output:

Story Views: 500

3. Imagine a Flipkart-like app: create an abstract class Product with an abstract method upload(). Then, create two subclasses, Electronics and Clothing, that each implement the upload() method to print a different upload message.

```
abstract class Product {
  abstract void upload();
}

class Electronics extends Product {
  void upload() {
    System.out.println("Electronics product uploaded to Flipkart.");
  }
}

class Clothing extends Product {
  void upload() {
    System.out.println("Clothing product uploaded to Flipkart.");
  }
}
```

```
public class Main {
  public static void main(String[] args) {
    Electronics e = new Electronics();
    Clothing c = new Clothing();

    e.upload();
    c.upload();
  }
}
```

Output:

Electronics product uploaded to Flipkart.

Clothing product uploaded to Flipkart.

**4. Refactor the following code so that the user's phone number in the UserProfile class is private and can only be set or retrieved using public methods.

Hint: Add private access modifier to the phone number and create setPhoneNumber() and getPhoneNumber() methods.**

```
class UserProfile {
  private String phoneNumber;

  public void setPhoneNumber(String phoneNumber) {
    this.phoneNumber = phoneNumber;
  }

  public String getPhoneNumber() {
    return phoneNumber;
```

```
  }
}

public class Main {
  public static void main(String[] args) {
    UserProfile user = new UserProfile();

    user.setPhoneNumber("9876543210");

    System.out.println("Phone Number: " + user.getPhoneNumber());
  }
}
```

Output:

Phone Number: 9876543210

Explanation: The phone number is private, so it can only be accessed or modified through the public getPhoneNumber() and setPhoneNumber() methods.

5. Explain in your own words the difference between encapsulation and abstraction, using examples from any social media app you use daily (like Instagram or WhatsApp).

Encapsulation	**Abstraction**
Encapsulation means wrapping data and methods into one class and controlling access to the data.	Abstraction means hiding implementation details and showing only essential features.
Example: WhatsApp keeps user profile information private and allows access through methods.	Example: WhatsApp lets us send messages without showing the internal process used to deliver them.
Achieved using classes and access modifiers like private.	Achieved using abstract classes and interfaces in Java.